COMPSCI 572.01 (NLP)
Pretraining 1

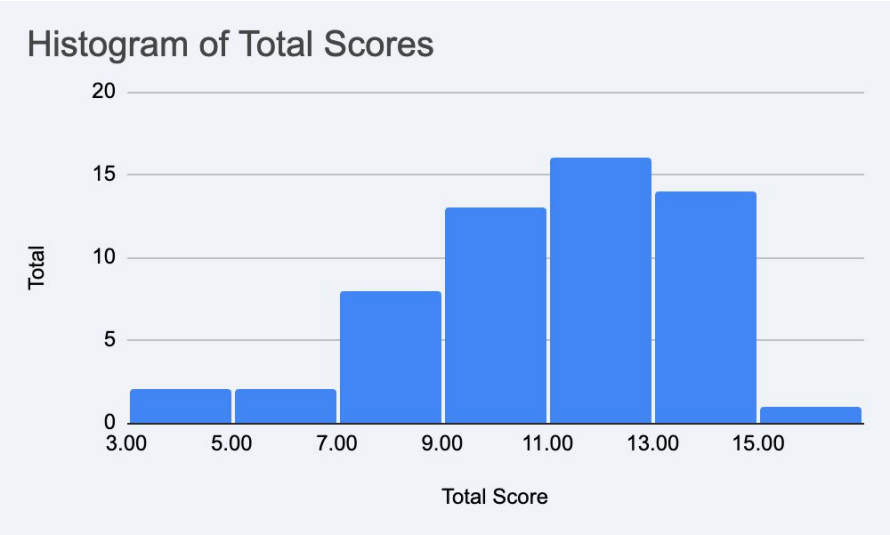
Slides inspired by John Hewitt at Stanford CS224N

Announcements

- Assignment 2 and Weekly exercise due tonight
- Assignment 3 will be released over the weekend
  - On information retrieval
  - Due Wednesday 3/25 11.59pm

Quiz 1 grades

- Mean: 10.77
- Median: 11.50



Overview

- ELMo: contextualized word embeddings
- Pretraining recipe
- Pretrained model architectures
  - Encoder only (BERT)
  - Decoder only (GPT)
  - Encoder-Decoder (T5)

Recap: Word vectors

Distributional Hypothesis: You shall know a word by the company it keeps (Firth, 1957)

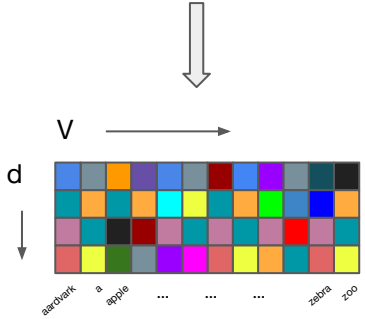
Recap: Word vectors

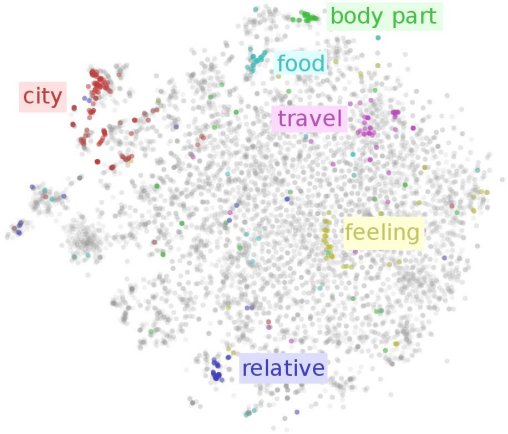
Distributional Hypothesis: You shall know a word by the company it keeps (Firth, 1957)

Words

Contexts

	1.	2.	3.	4.
Tesgüino	1	1	1	1
loud	0	0	0	0
tortillas	0	1	0	1
wine	1	1	1	0





What is missing from word vectors?

Distributional Hypothesis: You shall know a word by the company it keeps (Firth, 1957)

Also, Firth (1935):

"... the complete meaning of a word is always **contextual**, and no study of meaning apart from a complete context can be taken seriously."

1935 quote pointed out by Yoav Goldberg

What is missing from word vectors?

Many words have multiple senses:

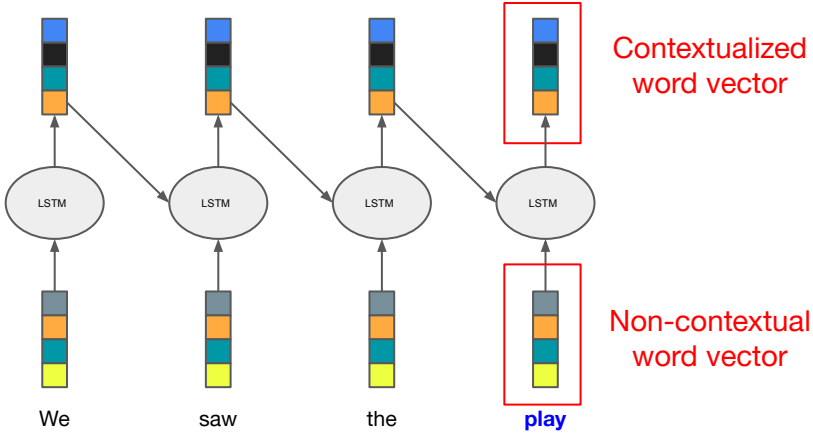
*We went to see the **play** Romeo and Juliet at the theater.*

*The coach devised a great **play** that put the visiting team on the defensive.*

*The children went out to **play** in the park.*

Idea: create a different word vector for every context in which **play** appears

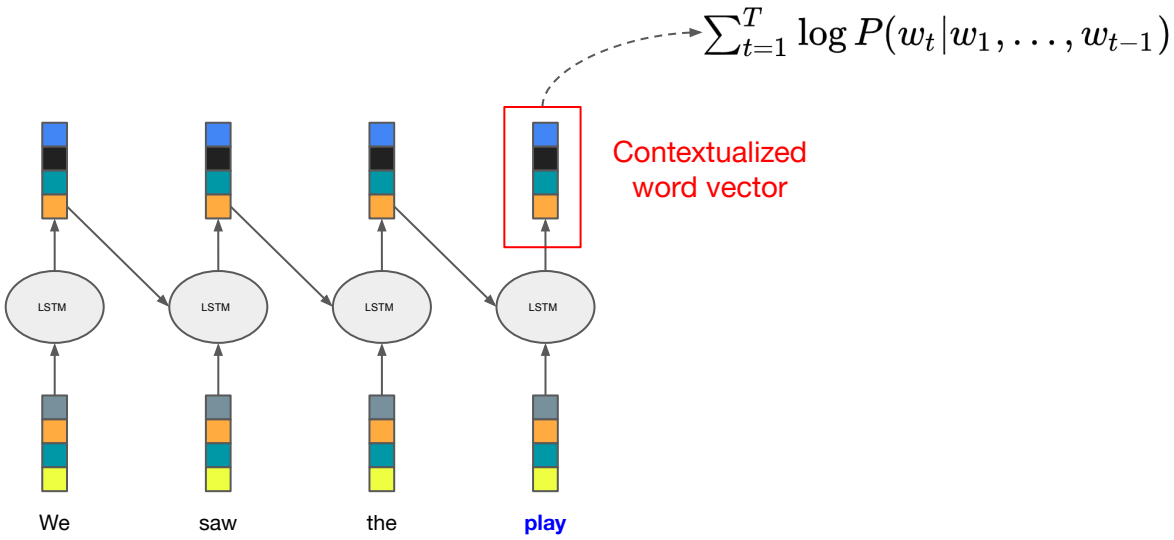
Contextualized word vectors



Idea: Treat the hidden states from an LSTM (or Transformer) model as contextualized word vectors

- What do we train the LSTM on?

ELMo: Embeddings from Language Models



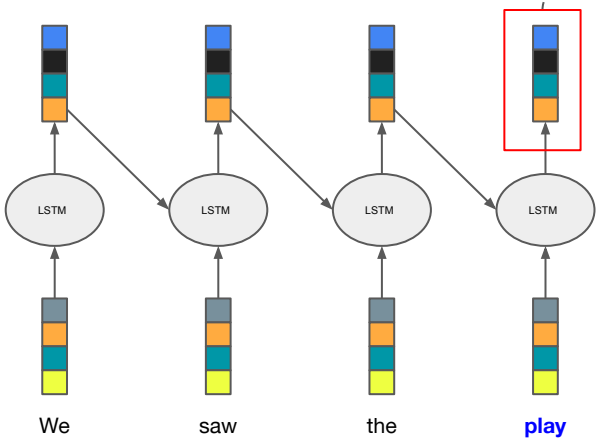
Idea: Treat the hidden states from an LSTM (or Transformer) model as contextualized word vectors

- What do we train the LSTM on? Language modeling!

Peters et al; 2018

ELMo: Embeddings from Language Models

$$\sum_{t=1}^{T} \log P(w_t | w_1, \ldots, w_{t-1})$$



Contextualized word vector

Problem: language modeling only uses the *left* context

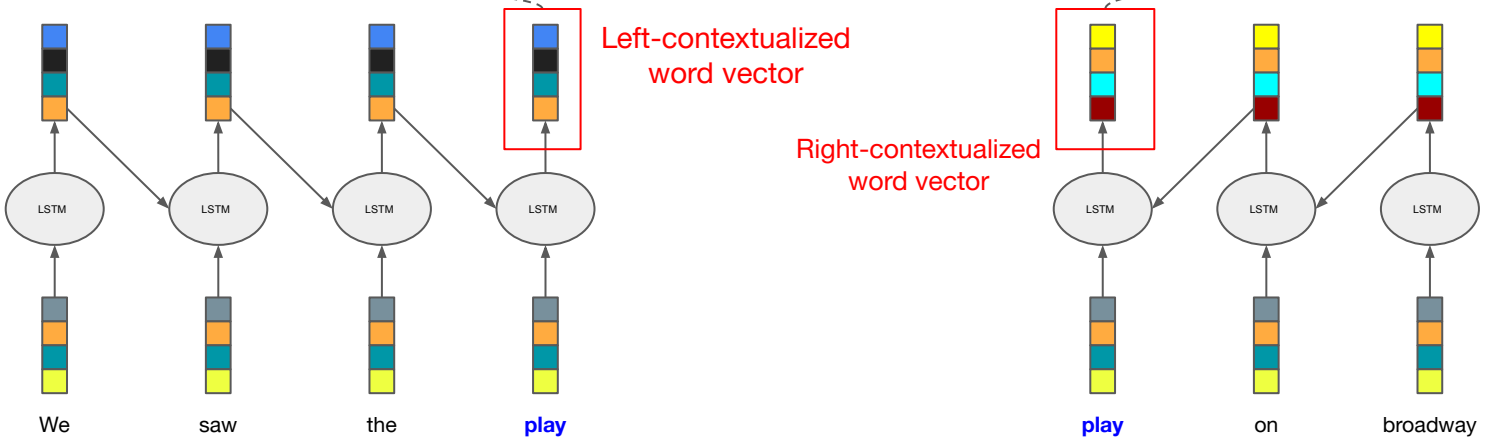
Idea: Treat the hidden states from an LSTM (or Transformer) model as contextualized word vectors

- What do we train the LSTM on? Language modeling!

Peters et al; 2018

ELMo: Embeddings from Language Models

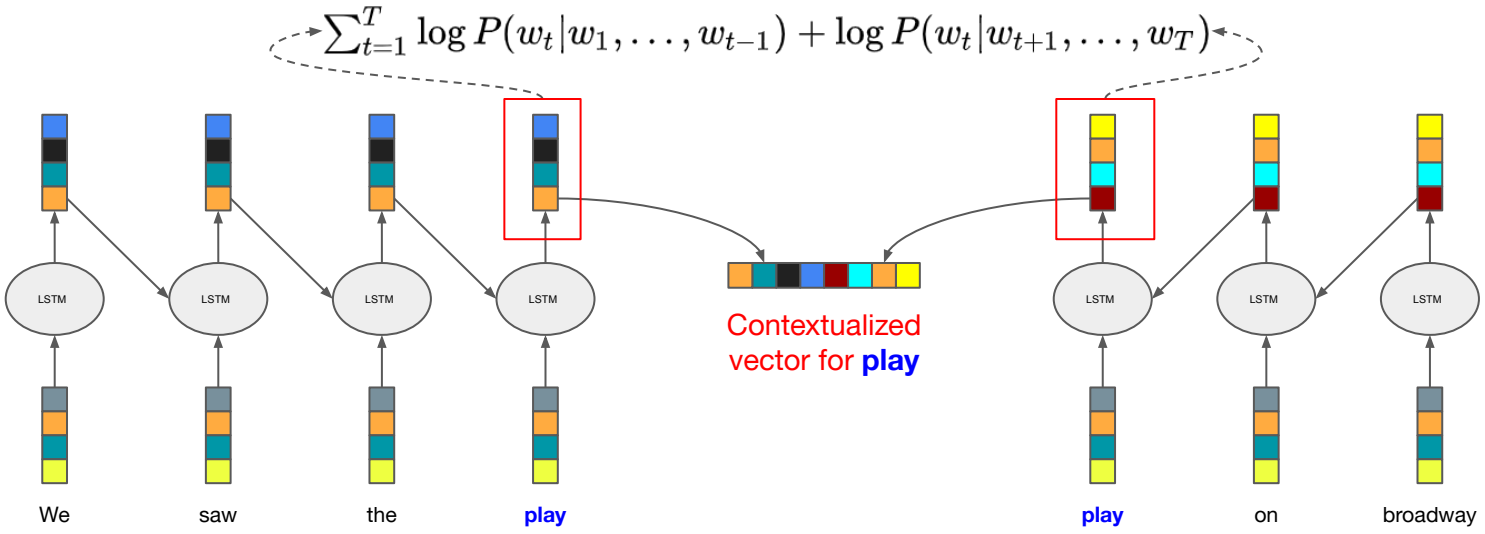
$$\sum_{t=1}^{T} \log P(w_t | w_1, \ldots, w_{t-1}) + \log P(w_t | w_{t+1}, \ldots, w_T)$$



- Use two language models -- from left-to-right and right-to-left

Peters et al; 2018

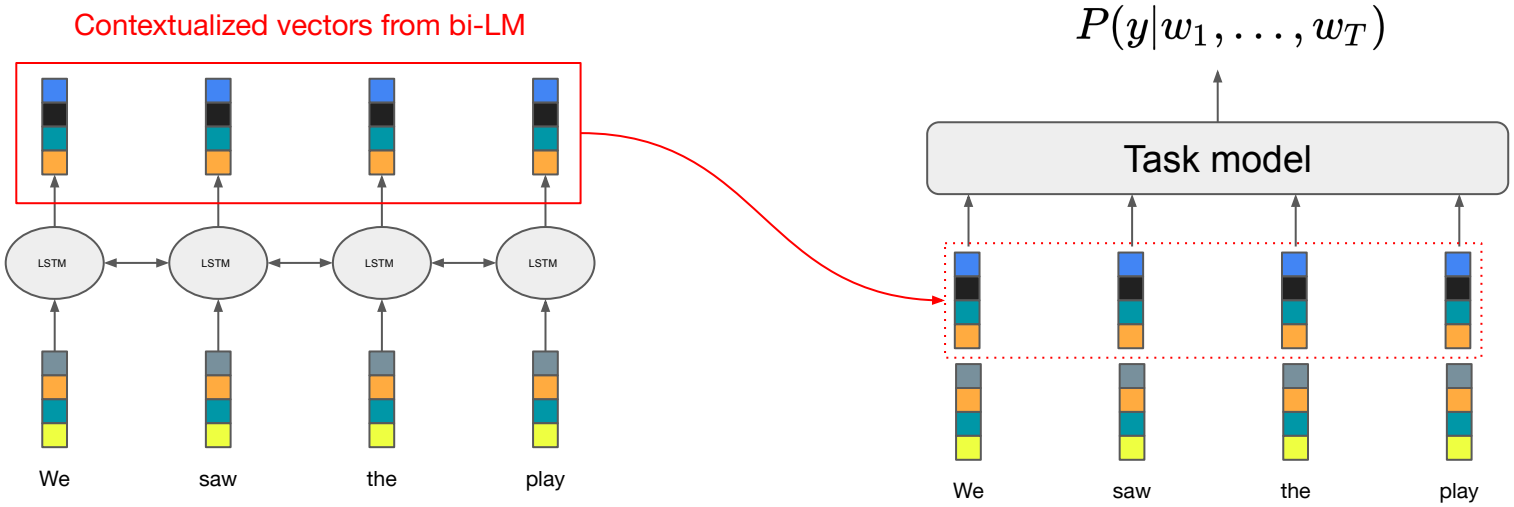
ELMo: Embeddings from Language Models



- Use two language models -- from left-to-right and right-to-left
- Also called a bi-LM (not the same as bi-LSTM)

Peters et al; 2018

Using ELMo for downstream tasks



Combine with non-contextual word embeddings

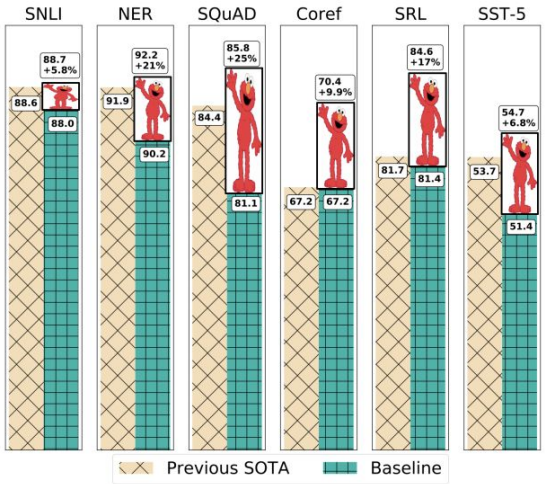
- Bi-LM parameters are kept fixed when training downstream network

Peters et al; 2018

Details

- Huge improvements across benchmark tasks:

  - SQuAD: question answering
  - SNLI: natural language inference
  - SRL: semantic role labeling
  - Coref: coreference resolution
  - NER: named entity recognition
  - SST-5: sentiment analysis

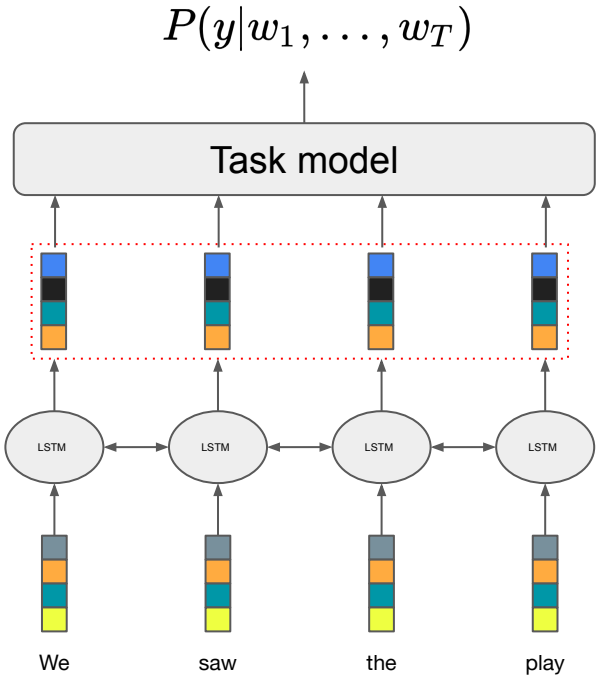


Gain due to ELMo embeddings

Peters et al; 2018

Why does ELMo help?

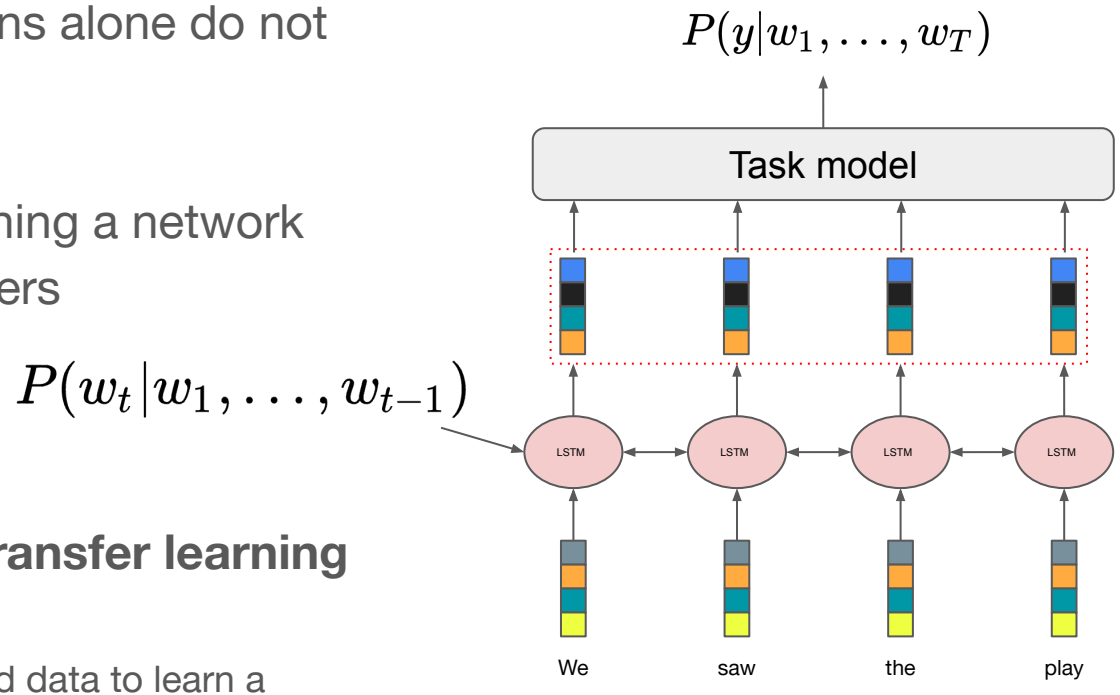
- Contextual representations alone do not explain this
- Effectively we're just training a network with additional LSTM layers



Peters et al; 2018

Why does ELMo help?

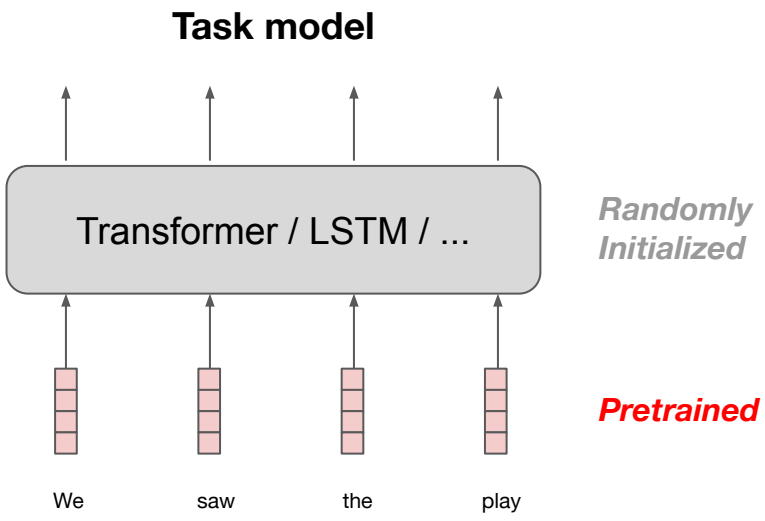
- Contextual representations alone do not explain this
- Effectively we're just training a network with additional LSTM layers
- ELMo is an instance of **transfer learning** or **pretraining**
  - We're leveraging unlabeled data to learn a deeper network with more layers



Peters et al; 2018

Where we are

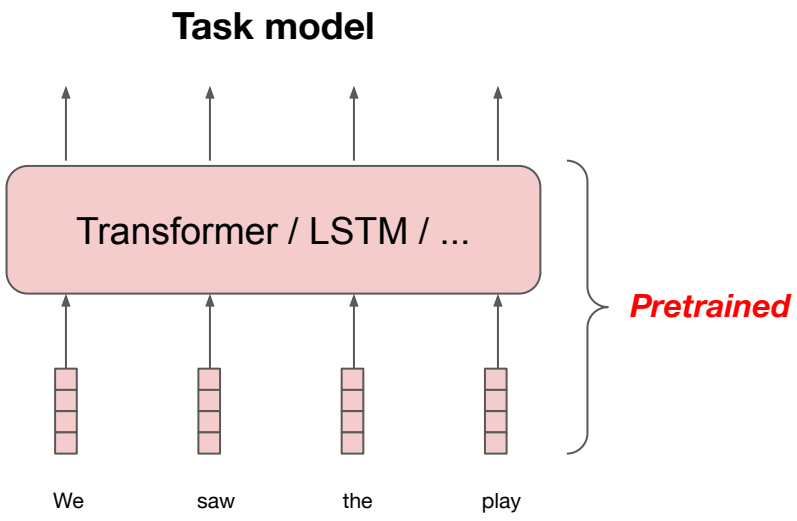
- ELMo improves over static word embeddings
- But most of the parameters in the network for the downstream task are still randomly initialized



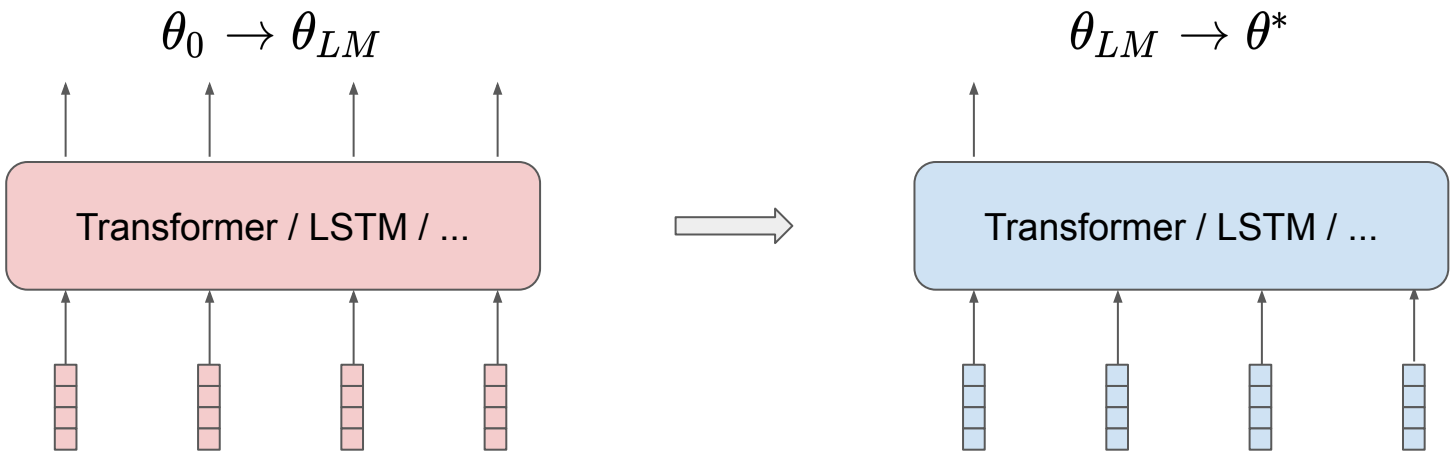
Where we are

- Since ELMo, many approaches have been developed for pretraining the entire network on unlabeled data

- These methods hide some part of the input and try to reconstruct it
  - Similar to language modeling



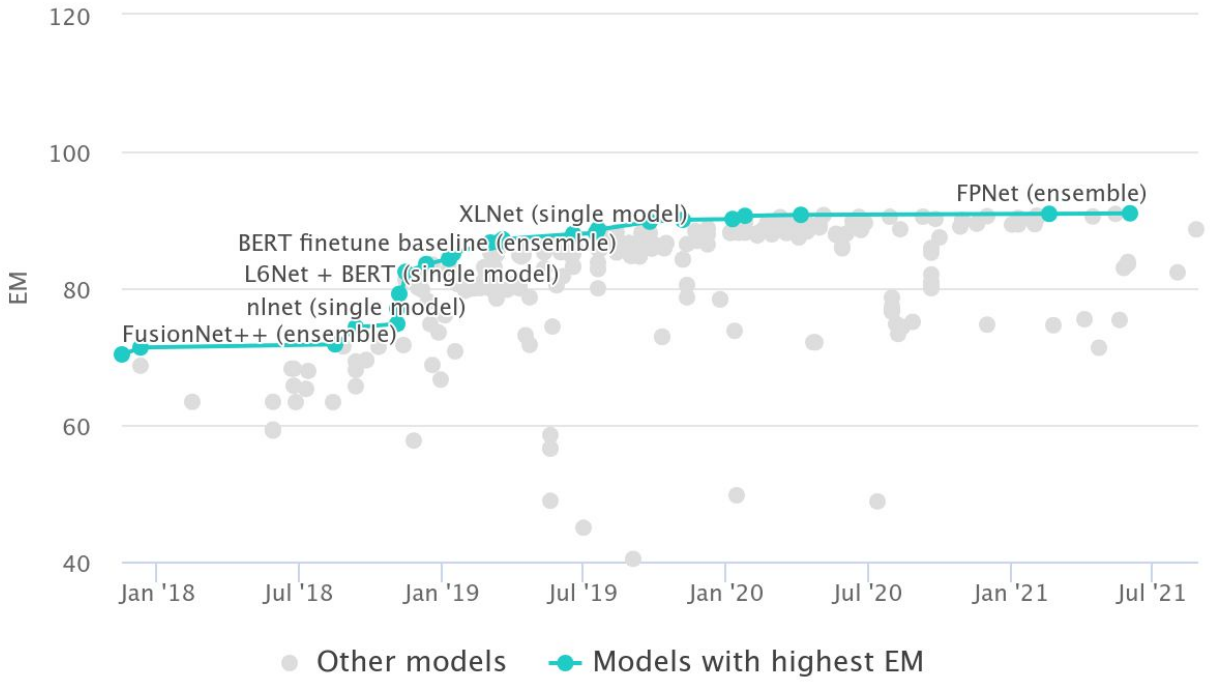
Pretraining: Recipe



1. **Pretrain** the network on language modeling or some other reconstruction objective over unlabeled data
2. **Finetune** the network on task-specific labeled data

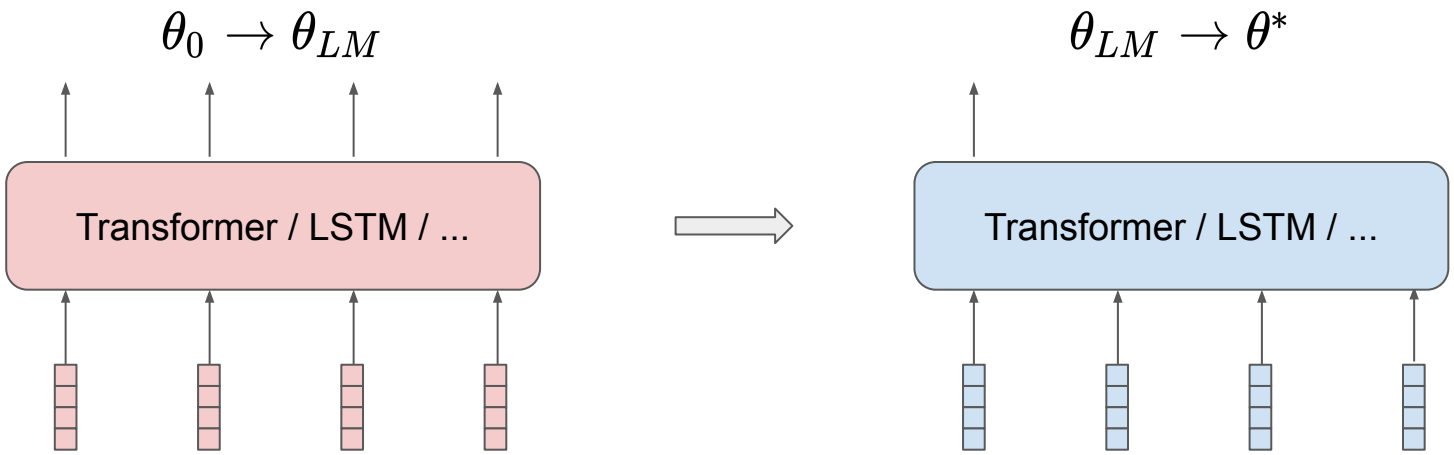
Pretraining had a transformative impact on NLP

State of the art on Squad2.0 – a benchmark for language understanding



https://paperswithcode.com/sota/question-answering-on-squad20

Pretraining: Recipe



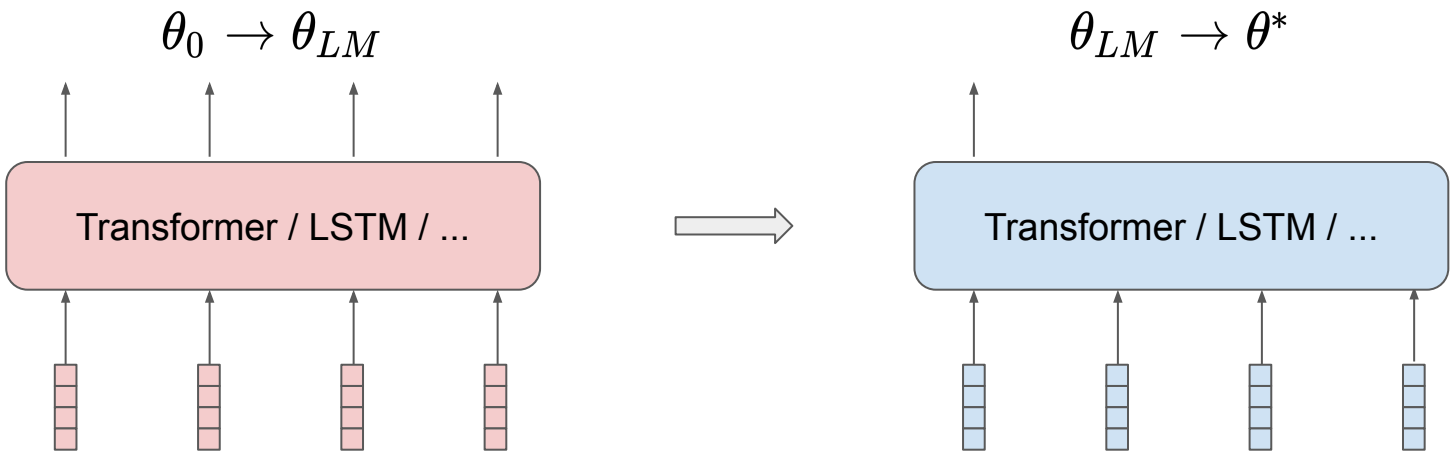
Why does this help?

- Linguistic intuition
  - Reconstructing inputs teaches the model a variety of language skills
  - Classification can often be formulated as sentence completion (Saunshi et al; 2021)

What can we learn from reconstructing the input?

- *Duke University is located in _____, North Carolina.* **Facts**
- *There is _____ man standing outside.* **Syntax**
- *John returned the suit because _____ did not like the color.*
- *John returned the suit because _____ was torn.* **Coreference / commonsense**
- *Excruciatingly long and boring, the movie was _____.* **Sentiment**

Pretraining: Recipe

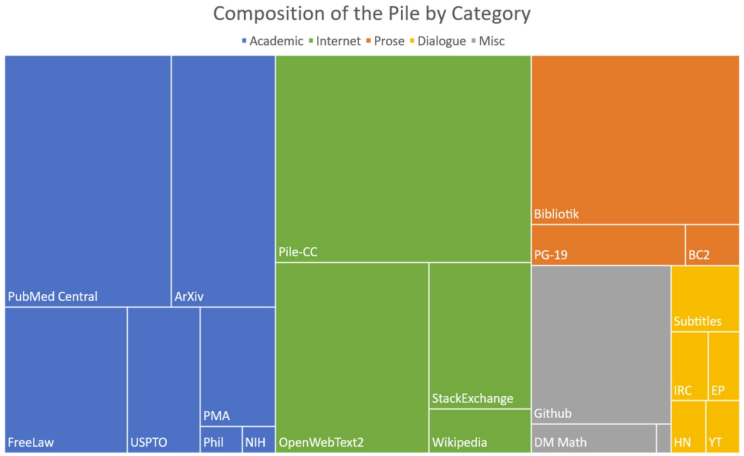


Why does this help?

- Optimization perspective: SGD will stick close to LM parameters -- local minima around LM parameters generalize better

Pretraining data

- We only need **unlabeled** data – the more the better
- Usually a mix of online sources



The Pile (Dec 2020)

Table 2: Token counts for the RedPajama-V1 dataset.

Dataset Slice	Token Count
CommonCrawl	878B
C4	175B
GitHub	59B
Books	26B
ArXiv	28B
Wikipedia	24B
StackExchange	20B
Total	1.2T

RedPajama-v1 (Weber et al, 2024)

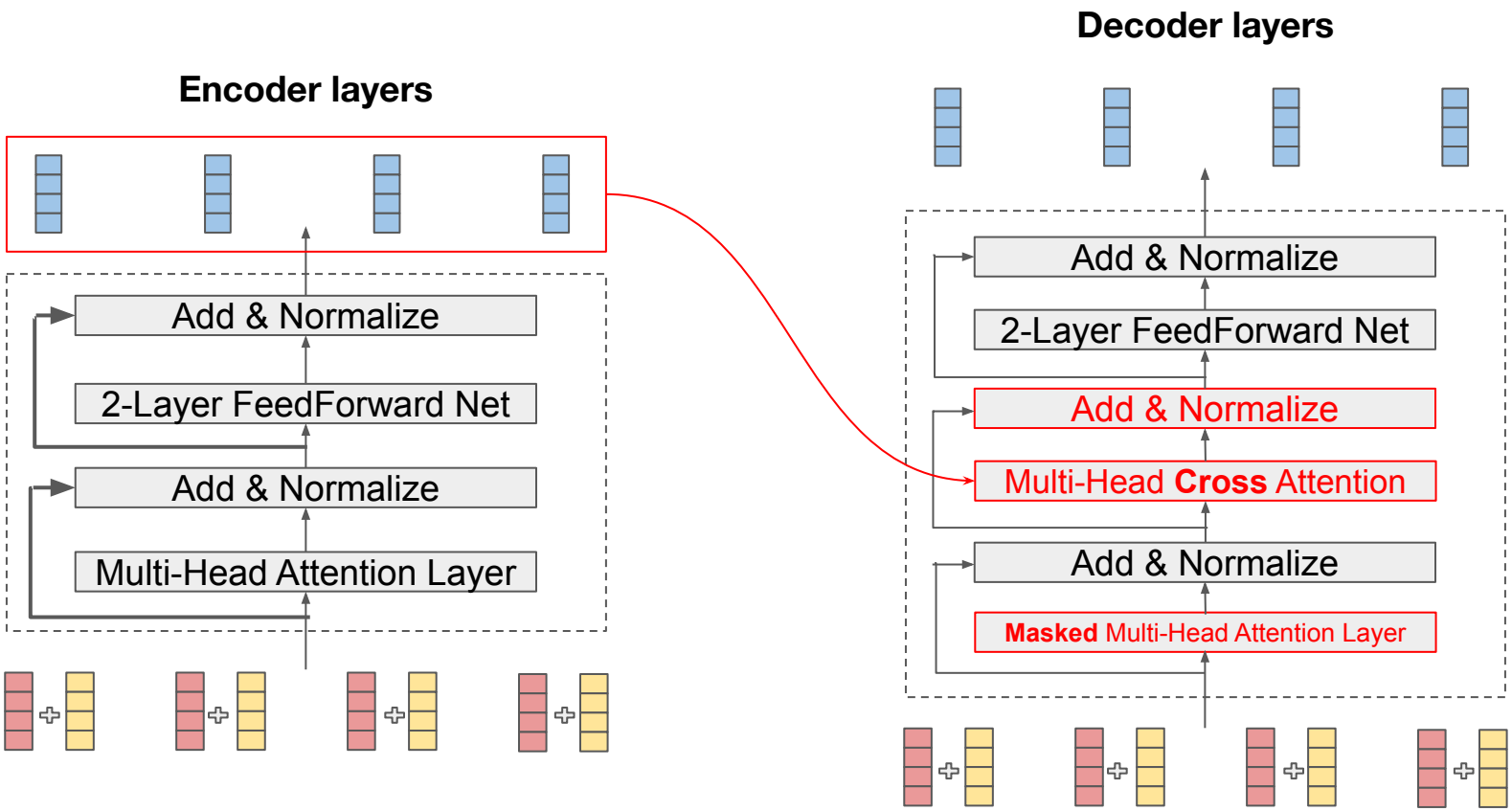
Pretraining data

- Internet data is noisy → deduplicate and then filter

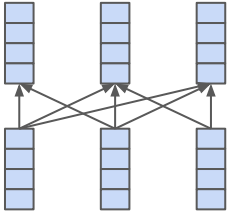
```python
def rpv1_rules_pass(sample) -> bool:
    """ function returns True if the sample complies with the filtering rules used in RP-V1 """
    signals = json.loads(sample["quality_signals"])

    # rule 1: the wikipedia reference classifier score must be higher than 0.25
    wikiref_score = signals["rps_doc_ml_wikiref_score"][0][2]
    if wikiref_score < 0.25:
        return False

    return True

def c4_rules_pass(sample) -> bool:
    """ function returns True if the sample complies with the filtering rules used in C4 """
    signals = json.loads(sample["quality_signals"])

    # rule 1: at least 3 sentences
    num_sentences = signals["rps_doc_num_sentences"][0][2]
    if num_sentences < 3:
        return False

    # rule 2: page may not contain bad words
    n_bad_words = signals["rps_doc_ldnoobw_words"][0][2]
    if n_bad_words > 0:
        return False

    # rule 3: page may not contain placeholder "lorem ipsum" text
    lorem_ipsum = signals["rps_doc_lorem_ipsum"][0][2]
    if lorem_ipsum > 0:
        return False

    # rule 4: ...

    return True
```

```python
def gopher_rules_pass(sample) -> bool:
    """ function returns True if the sample complies with Gopher rules """
    signals = json.loads(sample["quality_signals"])

    # rule 1: number of words between 50 and 10'000
    word_count = signals["rps_doc_word_count"][0][2]
    if word_count < 50 or word_count > 10_000:
        return False

    # rule 2: mean word length between 3 and 10
    mean_word_length = signals["rps_doc_mean_word_length"][0][2]
    if mean_word_length < 3 or mean_word_length > 10:
        return False

    # rule 2: symbol to word ratio below 0.1
    symbol_word_ratio = signals["rps_doc_symbol_to_word_ratio"][0][2]
    if  symbol_word_ratio > 0.1:
        return False

    # rule 3: 90% of lines need to start without a bullet point
    n_lines = signals["ccnet_nlines"][0][2]
    n_lines_bulletpoint_start = sum(map(lambda ln: ln[2], signals["rps_lines_start_with_bulletpoint"]))
    if n_lines_bulletpoint_start / n_lines > 0.9:
        return False

    # rule 4: the ratio between characters in the most frequent 2-gram and the total number
    # of characters must be below 0.2
    top_2_gram_frac = signals["rps_doc_frac_chars_top_2gram"][0][2]
    if top_2_gram_frac > 0.2:
        return False

    # rule 5: ...


    return True
```

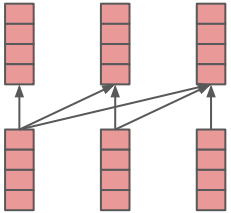
Recall: Transformer Encoder-Decoders



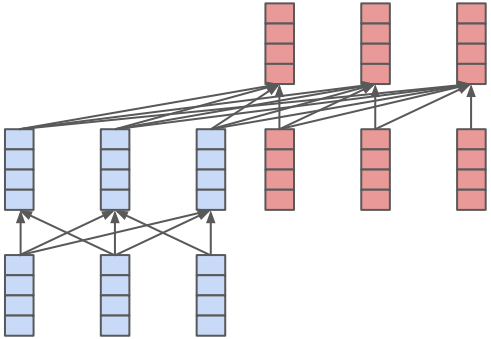
Model architectures for pretraining



Encoder only

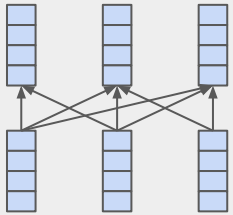


Decoder only



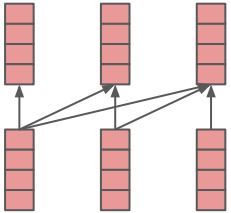
Encoder-Decoders

Model architectures for pretraining

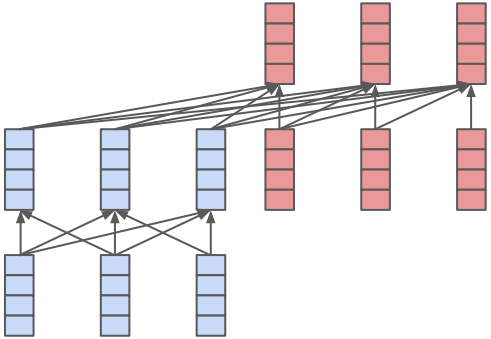


Encoder only

- Condition on both previous and future words
- What do we train these on?
- Cannot be used for generating text



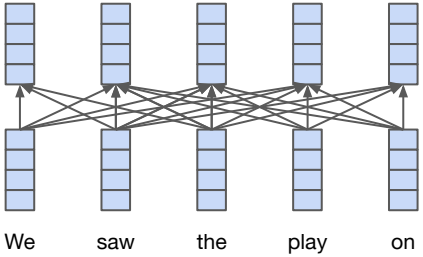
Decoder only



Encoder-Decoders

Encoder-only pretraining

- Encoder models receive **bidirectional context**
  - Hence we cannot use language modeling to train



Encoder-only pretraining

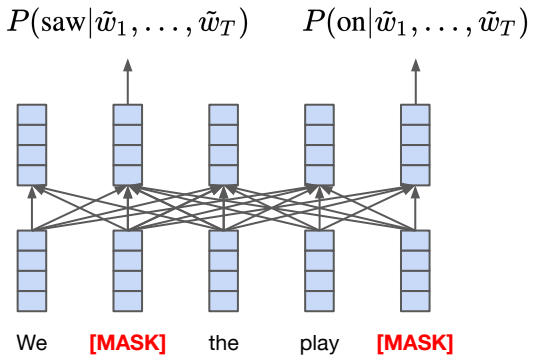
- Encoder models receive **bidirectional context**
  - Hence we cannot use language modeling to train

- Instead, use **masked language modeling**
  - Replace a fraction of input tokens with a special “[MASK]” token and predict them from the output

$$h_1, \ldots, h_T = \text{Encoder}(\tilde{w}_1, \ldots, \tilde{w}_T)$$

$$P(w_i | \tilde{w}_1, \ldots, \tilde{w}_T) = \text{softmax}(Wh_i + b)$$

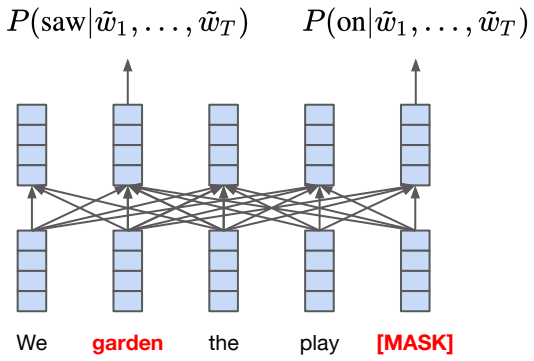
$$\text{loss} = -\sum_{i:\tilde{w}_i=[\text{MASK}]} \log P(w_i | \tilde{w}_1, \ldots, \tilde{w}_T)$$



Bidirectional Encoder Representations from Transformers (BERT)

- One of the first pretrained models
  - Hugely popular for a long time

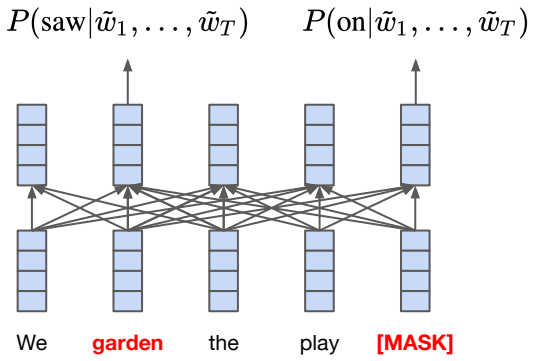
- Masking recipe: (over chunks of 512 tokens)
  - Take random 15% of the tokens
  - Replace 80% of these with [MASK]
  - Replace 10% of these with a random word
  - Keep 10% as they are (but still predict them)



Devlin et al; 2018

Bidirectional Encoder Representations from Transformers (BERT)

- One of the first pretrained models
  - Hugely popular for a long time
- Masking recipe: (over chunks of 512 tokens)
  - Take random 15% of the tokens
  - Replace 80% of these with [MASK]
  - Replace 10% of these with a random word
  - Keep 10% as they are (but still predict them)
- Why? So that the model has to build good representations of non-masked tokens as well



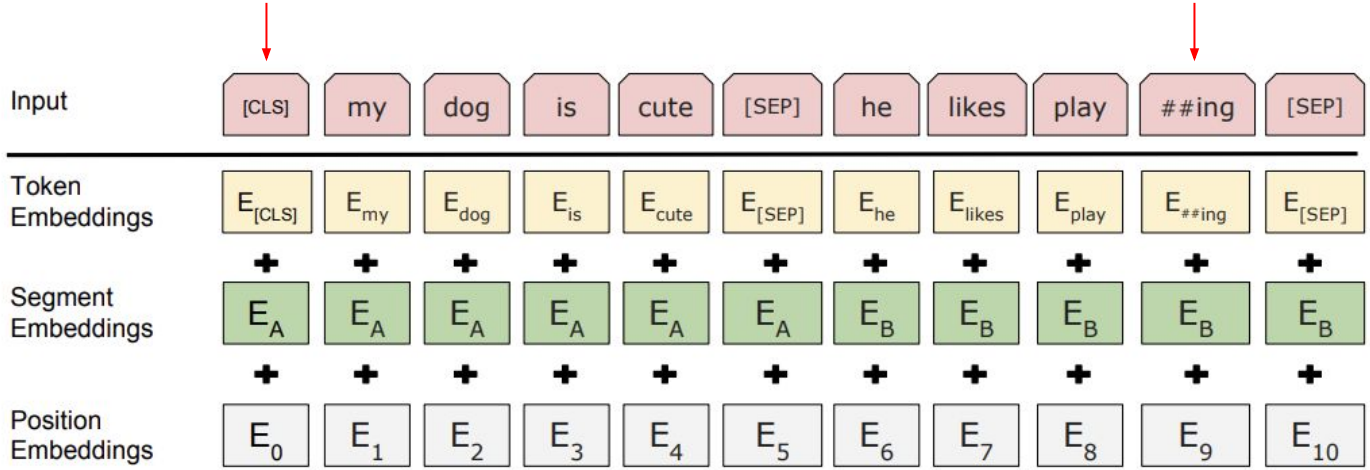
Devlin et al; 2018

Bidirectional Encoder Representations from Transformers (BERT)

- BERT also uses a next sentence prediction task
  - But recent work has argued this is not necessary

Uses final layer representation of [CLS] to predict whether two sentences are random / contiguous

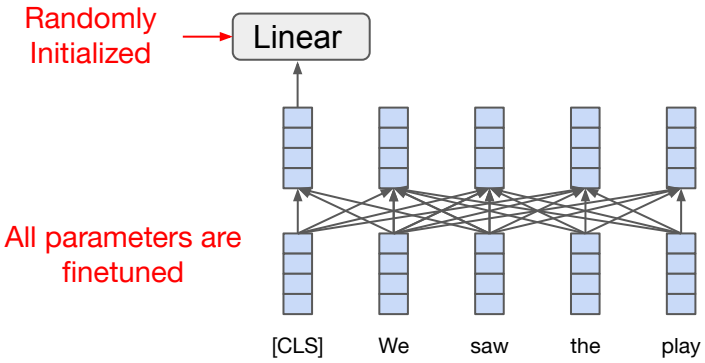
Subword pieces



Devlin et al; 2018

Finetuning BERT

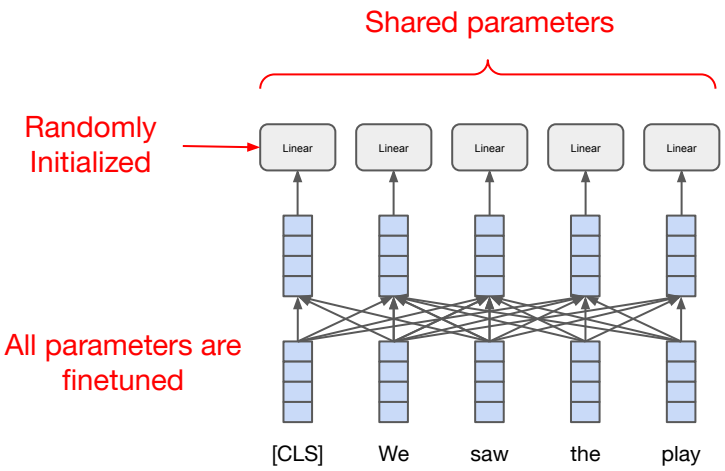
- Classification
  - Add a linear layer on top of [CLS] representation



Devlin et al; 2018

Finetuning BERT

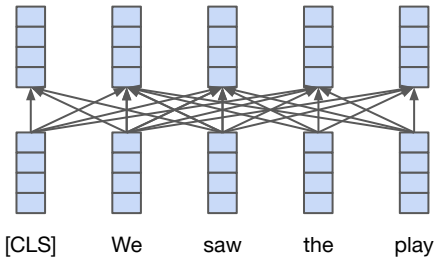
- Classification
  - Add a linear layer on top of [CLS] representation
- Sequence labeling
  - Add a linear layer on top of each token's representation
  - Works well even without CRF!



Devlin et al; 2018

Finetuning BERT

- Classification
  - Add a linear layer on top of [CLS] representation
- Sequence labeling
  - Add a linear layer on top of each token's representation
  - Works well even without CRF!
- Generation
  - No obvious way



Devlin et al; 2018

BERT Details

- Original paper: two sizes
  - BERT-base: 12 layers, 768d hidden sizes, 12 attention heads, 110M params
  - BERT-large: 24 layers, 1024d hidden sizes, 16 attention heads, 340M params

- Since then: 24 more smaller BERTs

	H=128	H=256	H=512	H=768
L=2	**2/128 (BERT-Tiny)**	2/256	2/512	2/768
L=4	4/128	**4/256 (BERT-Mini)**	**4/512 (BERT-Small)**	4/768
L=6	6/128	6/256	6/512	6/768
L=8	8/128	8/256	**8/512 (BERT-Medium)**	8/768
L=10	10/128	10/256	10/512	10/768
L=12	12/128	12/256	12/512	**12/768 (BERT-Base)**

https://github.com/google-research/bert

BERT Details

- Data: BooksCorpus + Wikipedia (2.5B words)
  - 90% training on chunks of 128 subword tokens
  - 10% training on chunks of 512 subword tokens

Devlin et al; 2018

BERT Details

- Data: BooksCorpus + Wikipedia (2.5B words)
  - 90% training on chunks of 128 subword tokens
  - 10% training on chunks of 512 subword tokens
- Pretraining is expensive (4 days on 16 cloud TPUs)
- But finetuning is cheap (~1 hour on GPU with enough memory)

Devlin et al; 2018

BERT Results

System	MNLI-(m/mm)	QQP	QNLI	SST-2	CoLA	STS-B	MRPC	RTE	**Average**
	392k	363k	108k	67k	8.5k	5.7k	3.5k	2.5k	-
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
$\text{BERT}_{\text{BASE}}$	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
$\text{BERT}_{\text{LARGE}}$	**86.7/85.9**	**72.1**	**92.7**	**94.9**	**60.5**	**86.5**	**89.3**	**70.1**	**82.1**

With pretrained models, larger sizes are almost always better

Devlin et al; 2018

RoBERTa: Robustly Optimized BERT

- Train for longer with much larger batch sizes
  - And remove next sentence prediction
- Add more data

Model	data	bsz	steps	SQuAD (v1.1/2.0)	MNLI-m	SST-2
RoBERTa						
with BOOKS + WIKI	16GB	8K	100K	93.6/87.3	89.0	95.3
+ additional data (§3.2)	160GB	8K	100K	94.0/87.7	89.3	95.6
+ pretrain longer	160GB	8K	300K	94.4/88.7	90.0	96.1
+ pretrain even longer	160GB	8K	500K	**94.6/89.4**	**90.2**	**96.4**
$\text{BERT}_{\text{LARGE}}$						
with BOOKS + WIKI	13GB	256	1M	90.9/81.8	86.6	93.7

Liu et al; 2019

Masking variants

- Random masking
  - Take random 15% subword tokens

Di Caprio ##’s acting in Inception was inc ##red ##ible

Di [MASK] ##’s acting in Inception was inc [MASK] ##ible

Masking variants

- Random masking
  - Take random 15% subword tokens
- Whole word masking
  - Mask all subword tokens in the same word together

Di Caprio ##’s acting in Inception was inc ##red ##ible

Di Caprio ##’s acting in Inception was [MASK] [MASK] [MASK]

Masking variants

- Random masking
  - Take random 15% subword tokens
- Whole word masking
  - Mask all subword tokens in the same word together
- Span masking (SpanBERT; Joshi et al, 2019)
  - Mask contiguous multi-word spans together

Di Caprio ##’s acting in Inception was inc ##red ##ible

Di Caprio ##’s [MASK] [MASK] [MASK] was inc ##red ##ible

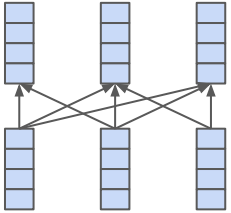
Masking variants

- Random masking
  - Take random 15% subword tokens
- Whole word masking
  - Mask all subword tokens in the same word together
- Span masking (SpanBERT; Joshi et al, 2019)
  - Mask contiguous multi-word spans together
- Salient span masking (SSM, Guu et al, 2020)
  - Mask named entities and dates

Di Caprio ##’s acting in Inception was inc ##red ##ible

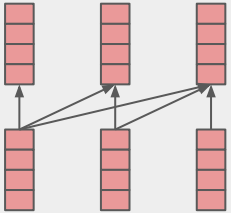
[MASK] [MASK] [MASK] acting in [MASK] was inc ##red ##ible

Model architectures for pretraining



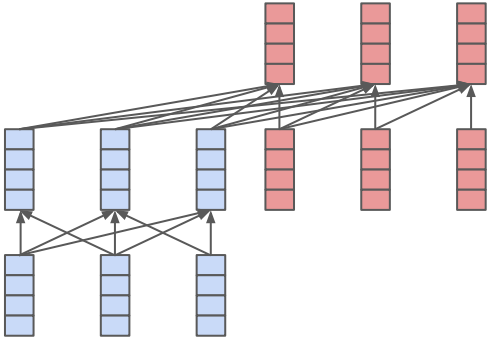
Encoder only

- Condition on both previous and future words
- What do we train these on?
- Cannot be used for generating text



Decoder only

- Only condition on previous words
- Training objective: language modeling
- Useful for generating text



Encoder-Decoders

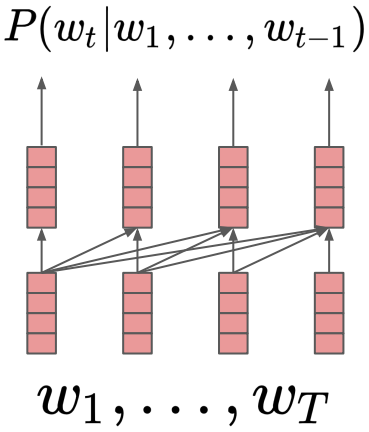
Decoder-only pretraining

- Pretrain on language modeling

$$h_1, \ldots, h_T = \text{Decoder}(w_1, \ldots, w_T)$$

$$P(w_t | w_1, \ldots, w_{t-1}) = \text{softmax}(W h_{t-1} + b)$$

$$\text{loss} = -\sum_{t=1}^{T} \log P(w_t | w_1, \ldots, w_{t-1})$$

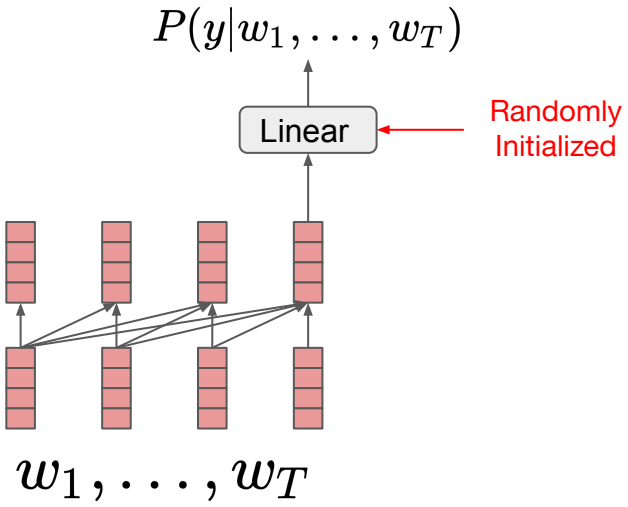


Decoder-only pretraining

- Pretrain on language modeling
- For classification: add a linear classifier on top of last word's hidden state

$$h_1, \ldots, h_T = \text{Decoder}(w_1, \ldots, w_T)$$

$$P(y|w_1, \ldots, w_T) = \text{softmax}(Wh_T + b)$$

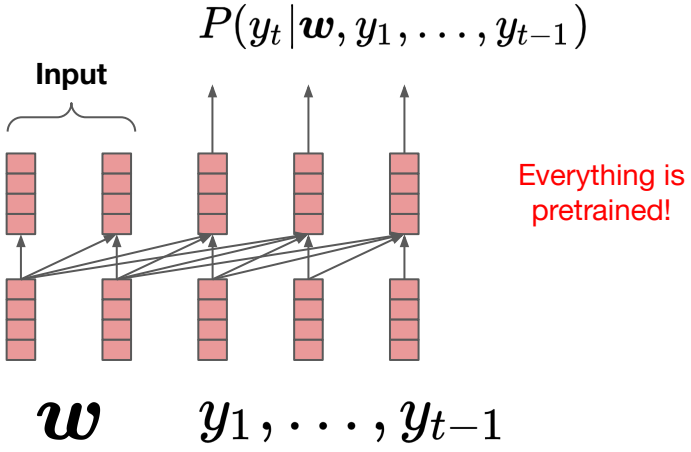


Decoder-only pretraining

- Pretrain on language modeling
- For classification: add a linear classifier on top of last word's hidden state
- For generation: continue training as a conditional language model

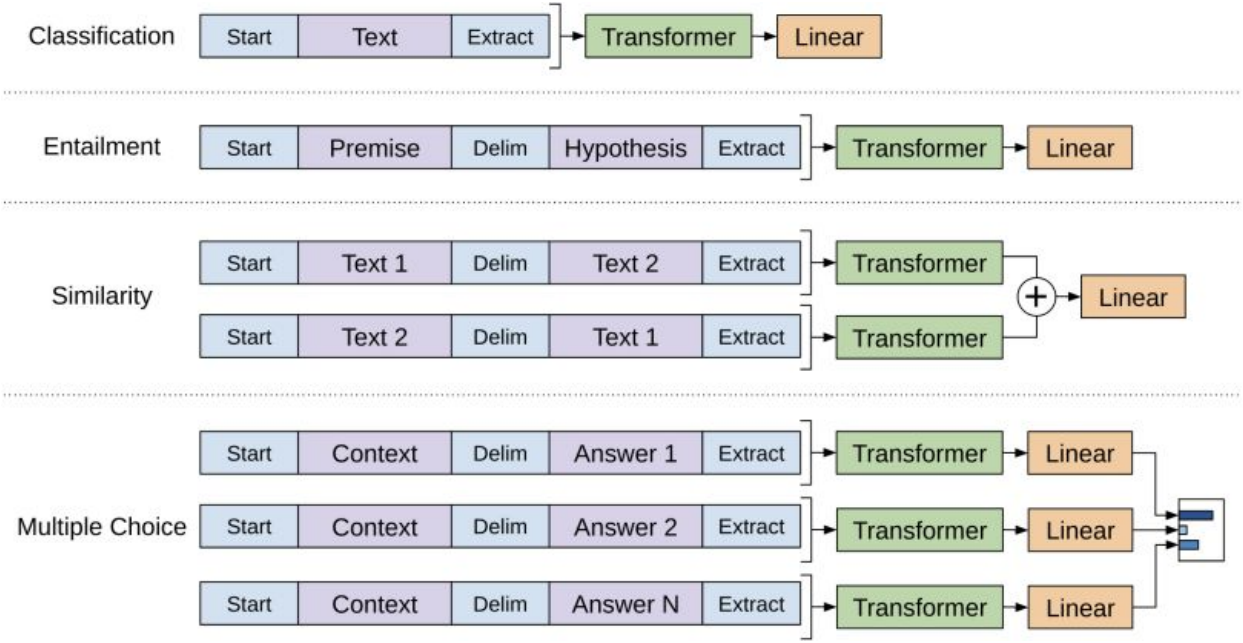
$$h_1, \ldots, h_t = \text{Decoder}(\boldsymbol{w}, y_1, \ldots, y_t)$$

$$P(y_t | \boldsymbol{w}, y_1, \ldots, y_{t-1}) = \text{softmax}(Wh_t + b)$$



Decoder-only pretraining

Constructing the input for finetuning:



Generative Pretrained Transformer, Radford et al; 2018

GPT-1

- Model: 12 layers, 768-dim hidden states, 3072-dim feedforward layers, 12 attention heads
- Data: BooksCorpus (7000 books, 800M words)

System	MNLI-(m/mm)	QQP	QNLI	SST-2	CoLA	STS-B	MRPC	RTE	**Average**
	392k	363k	108k	67k	8.5k	5.7k	3.5k	2.5k	-
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
$\text{BERT}_{\text{BASE}}$	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
$\text{BERT}_{\text{LARGE}}$	**86.7/85.9**	**72.1**	**92.7**	**94.9**	**60.5**	**86.5**	**89.3**	**70.1**	**82.1**

- Worse than BERT on classification, but can be used for generation!

Radford et al; 2018

GPT-2

- Model: 1.5B params, 48 layers, 1600-dim hidden states
- Data: WebText (outbound links from Reddit), 40GB of text
- First model to produce convincing samples of natural language:

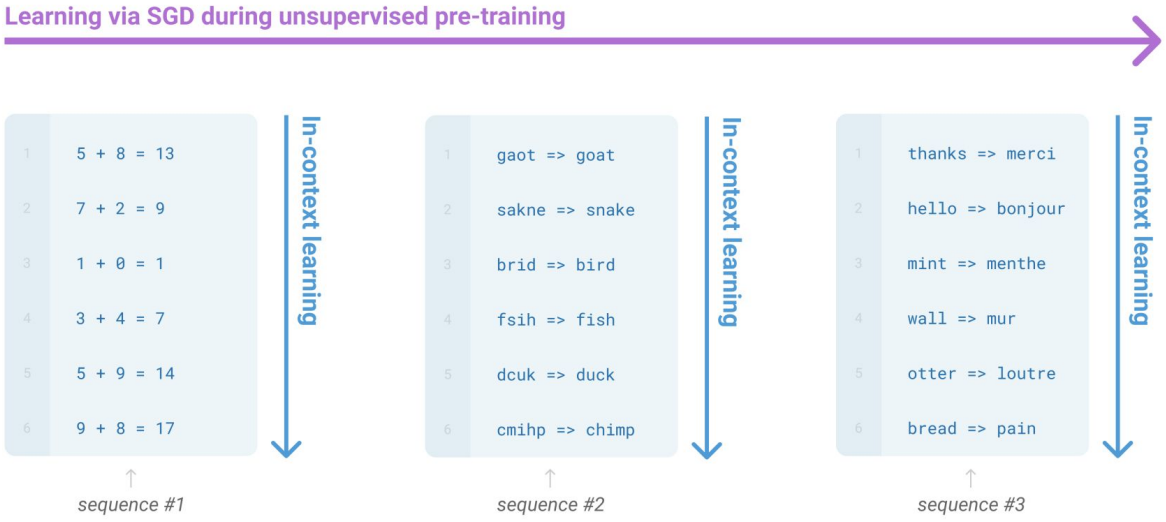
Context (human-written): In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.
GPT-2: The scientist named the population, after their distinctive horn, Ovid's Unicorn. These four-horned, silver-white unicorns were previously unknown to science. Now, after almost two centuries, the mystery of what sparked this odd phenomenon is finally solved. Dr. Jorge Pérez, an evolutionary biologist from the University of La Paz, and several companions, were exploring the Andes Mountains when they found a small valley, with no other animals or humans. Pérez noticed that the valley had what appeared to be a natural fountain, surrounded by two peaks of rock and silver snow.

Language Models are Unsupervised Multitask Learners (OpenAI)

GPT-3

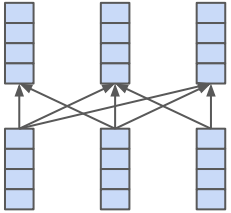
- Model: 175B params, 96 layers, 96 heads, 12K-dim hidden states
- Data: 500B tokens of text (Common Crawl + WebText + Wikipedia + Books)
- First model to show in-context learning (without gradient steps)

Not publicly available :(



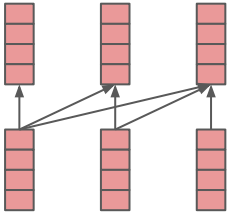
Language models are few-shot learners (May, 2020)

Model architectures for pretraining



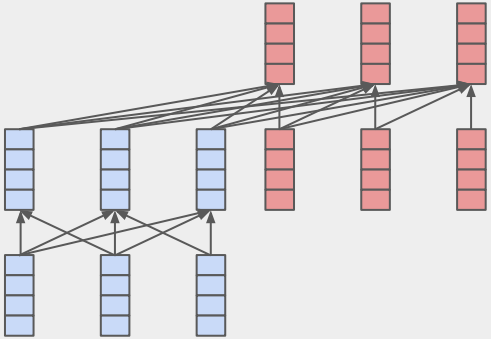
Encoder only

- Condition on both previous and future words
- What do we train these on?
- Cannot be used for generating text



Decoder only

- Only condition on previous words
- Training objective: language modeling
- Useful for generating text



Encoder-Decoders

- Best of both?
- What is the best way to train them?

Encoder-Decoder Pretraining

We can use language modeling: For every input,

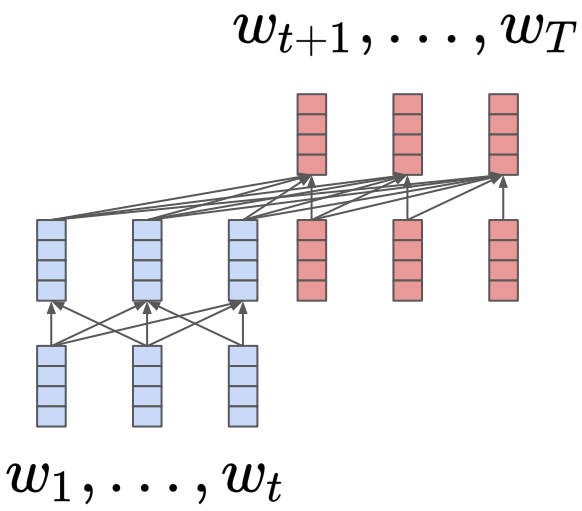
- Encode a prefix using the bidirectional encoder
- Predict the rest using the unidirectional decoder

$$h_1, \ldots, h_t = \text{Encoder}(w_1, \ldots, w_t)$$

$$h_{t+1}, \ldots, h_T = \text{Decoder}(w_{t+1}, \ldots, w_T, h_1, \ldots, h_t)$$

$$P(w_{t'} | w_{<t'}) = \text{softmax}(W h_{t'} + b)$$

$$\text{loss} = -\sum_{t'=t+1}^{T} \log P(w_{t'} | w_{<t'})$$



Exploring the limits of text-to-text transfer learning Raffel et al; 2019

Encoder-Decoder Pretraining

Or we can again use *masked* language modeling (a.k.a *Denoising*)

- Replace spans in the input with special tokens
- Predict those spans at the output

Original text

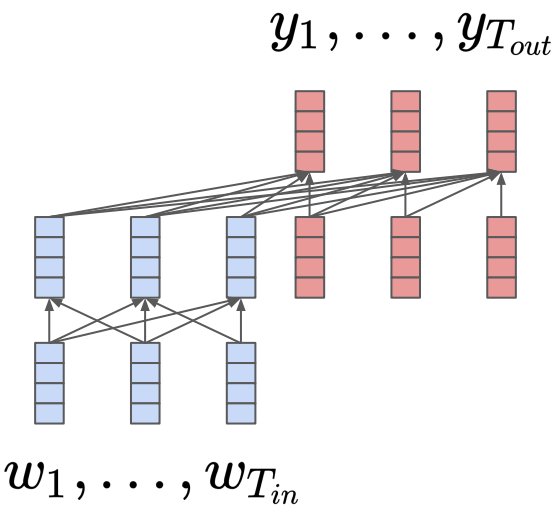
Thank you ~~for inviting~~ me to your party ~~last~~ week.

Inputs

Thank you <X> me to your party <Y> week.

Targets

<X> for inviting <Y> last <Z>

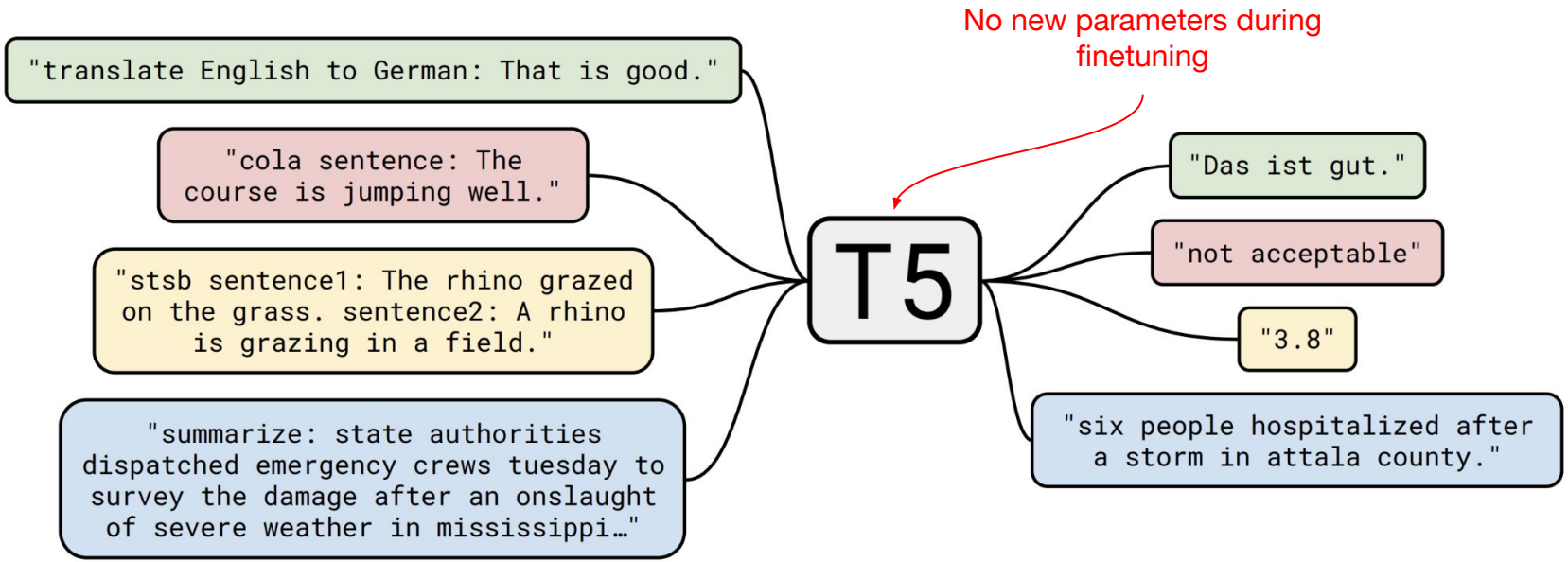


Exploring the limits of text-to-text transfer learning Raffel et al; 2019

Finetuning

Text-To-Text Transfer Transformer (T5)

- Formulate all tasks as a sequence-to-sequence problem



Exploring the limits of text-to-text transfer learning Raffel et al; 2019

Pretraining objective

A benchmark collection of NLP tasks

Architecture	Objective	Params	Cost	GLUE
★ Encoder-decoder	Denoising	$2P$	M	**83.28**
Enc-dec, shared	Denoising	P	M	82.81
Enc-dec, 6 layers	Denoising	P	$M/2$	80.88
Language model	Denoising	P	M	74.70
Prefix LM	Denoising	P	M	81.82
Encoder-decoder	LM	$2P$	M	79.56
Enc-dec, shared	LM	P	M	79.60
Enc-dec, 6 layers	LM	P	$M/2$	78.67
Language model	LM	P	M	73.78
Prefix LM	LM	P	M	79.68

Denoising (masked LM) works better than pure LM

Exploring the limits of text-to-text transfer learning Raffel et al; 2018

Model architecture

A benchmark collection of NLP tasks

Having separate encoders and decoders works best

Architecture	Objective	Params	Cost	GLUE
★ Encoder-decoder	Denoising	$2P$	M	**83.28**
Enc-dec, shared	Denoising	P	M	82.81
Enc-dec, 6 layers	Denoising	P	$M/2$	80.88
Language model	Denoising	P	M	74.70
Prefix LM	Denoising	P	M	81.82
Encoder-decoder	LM	$2P$	M	79.56
Enc-dec, shared	LM	P	M	79.60
Enc-dec, 6 layers	LM	P	$M/2$	78.67
Language model	LM	P	M	73.78
Prefix LM	LM	P	M	79.68

Exploring the limits of text-to-text transfer learning Raffel et al; 2019

Text-to-Text Transfer Transformer (T5)

- Five model sizes
  - T5-11B: 24 layers, 1024-dim hidden states, 65K-dim feedforward states, 128 attention heads

Exploring the limits of text-to-text transfer learning Raffel et al; 2019

Text-to-Text Transfer Transformer (T5)

- Five model sizes
  - T5-11B: 24 layers, 1024-dim hidden states, 65K-dim feedforward states, 128 attention heads
- On classification tasks:

Model	GLUE Average
Previous best	89.4[a]
T5-Small (60M)	77.4
T5-Base (220M)	82.7
T5-Large (770M)	86.4
T5-3B	88.5
T5-11B	**90.3**

BERT-base (110M)	79.6
BERT-large (340M)	82.1

Exploring the limits of text-to-text transfer learning Raffel et al; 2019

Text-to-Text Transfer Transformer (T5)

- Five model sizes
  - T5-11B: 24 layers, 1024-dim hidden states, 65K-dim feedforward states, 128 attention heads
- On sequence-to-sequence tasks:

	Machine Translation			Summarization		
Model	WMT EnDe BLEU	WMT EnFr BLEU	WMT EnRo BLEU	CNN/DM ROUGE-1	CNN/DM ROUGE-2	CNN/DM ROUGE-L
Previous best	**33.8**[e]	**43.8**[e]	**38.5**[f]	43.47[g]	20.30[g]	40.63[g]
T5-Small	26.7	36.0	26.8	41.12	19.56	38.35
T5-Base	30.9	41.2	28.0	42.05	20.34	39.40
T5-Large	32.0	41.5	28.1	42.50	20.68	39.75
T5-3B	31.8	42.6	28.2	42.72	21.02	39.94
T5-11B	32.1	43.4	28.1	**43.52**	**21.55**	**40.69**

Exploring the limits of text-to-text transfer learning Raffel et al; 2019

That’s it!

Next lecture: Multilingual models and scaling.